Informatika és távközlés ágazat

Synk

Készítette
Erdős Martin
Makkai Krisztián
Szedlák Krisztián
Szoftverfejlesztő és -tesztelő szak

Témavezető
Kerényi Róbert Nándor
oktató

Miskolc, 2026

Tartalomjegyzék

1. Bevezetés **4**

1.1. A projekt bemutatása és célja 4

1.2. Célközönség meghatározása 5

1.3. Alkalmazott technológiák és eszközök 5

2. Követelményanalízis **7**

2.1. Funkcionális követelmények 7

2.1.1. Webes felület (Next.js) 7

2.1.2. Mobil alkalmazás (React Native + Expo) 7

2.1.3. Adminisztrációs kliens (WPF) 7

2.2. Nem funkcionális követelmények 8

2.3. Használati esetek (Use Cases) 8

2.3.1. Kiemelt Use Case: Jegyvásárlás (UC-01) 8

3. Rendszertervezés **9**

3.1. Adatbázis terv 9

3.1.1. ER-diagram és kapcsolatok 9

3.1.2. Adatszótár és táblaleírások 10

3.2. Rendszerarchitektúra és komponensek 12

3.2.1. Általános architektúra 12

3.2.2. Komponensek és felelősségek 12

3.2.3. Skálázás és rendelkezésre állás 13

3.3. ORM (Object-Relational Mapping) 13

3.3.1. Tervezési döntések 13

3.3.2. Mapping és DTO stratégia 13

3.4. Entity Framework 14

3.4.1. DbContext és konfiguráció 14

3.4.2. Teljesítmény- és karbantartási praktikák 14

3.5. REST API végpontok tervezése 14

3.5.1. Erőforrás-orientált tervezés 14

3.5.2. Példa végpontok és viselkedés 14

3.5.3. Hiba- és státuszkódok 15

3.5.4. Biztonság és adatvédelem 15
3.6. UI/UX Tervek (Wireframes) 16

4. Megvalósítás **17**
4.1. Adatbázis réteg implementációja 17
4.2. REST Backend fejlesztése 17
4.2.1. Adatkezelés és Üzleti logika 17
4.2.2. Biztonság és Autentikáció (JWT) 17
4.3. React Frontend fejlesztése 17
4.3.1. Komponens-architektúra 17
4.3.2. Reszponzív megoldások és Mobil nézet 17
4.4. WPF Desktop alkalmazás 17
4.4.1. MVVM architektúra és Adatkötés 17
4.4.2. Törzsadat-karbantartó modulok 17

5. Minőségbiztosítás és Tesztelés **18**
5.1. Unit tesztek (Egységtesztelés) 18
5.2. API tesztelés (Postman/Swagger) 18
5.3. Felhasználói felület és Reszponzivitás tesztelése 18
5.4. Hibajegyzék és javítások 18

6. Felhasználói útmutató **19**
6.1. Rendszerkövetelmények és Telepítés 19
6.2. A szoftver használata (Web/WPF) 19

7. Összegzés **20**
7.1. Projekt értékelése 20
7.2. Továbbfejlesztési lehetőségek 20

Mellékletek **22**

Irodalomjegyzék **24**

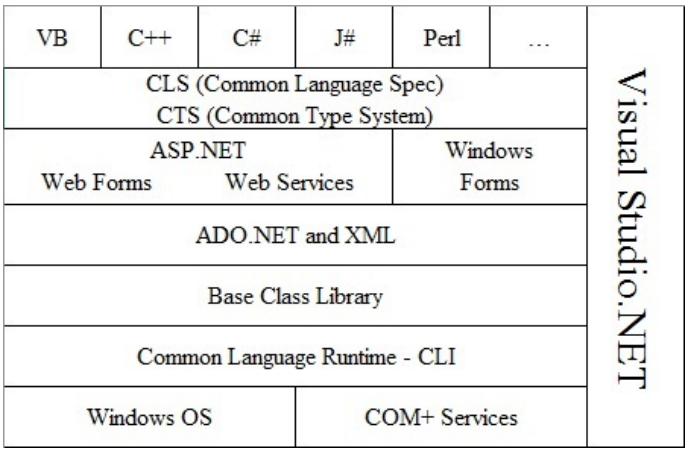
1. fejezet

Bevezetés

1.1. A projekt bemutatása és célja

A **Synk** egy innovatív, többplatformos koncertjegy-értékesítési ökoszisztéma. A projekt célja egy olyan integrált környezet létrehozása, amely a modern webes technológiák, az asztali alkalmazások ereje és a mobilis megoldások rugalmassága révén szolgálja ki a rendezvényszervezői piacot. A rendszer alapvető küldetése a jegyvásárlási folyamat transzparenssé tétele, a valós idejű készletkezelés és a biztonságos, digitális jegyfelhasználás biztosítása.

A Synk architektúrája négy fő pillérre épül: egy villámgyors vásárlói webfelületre, egy robusztus adminisztrációs asztali alkalmazásra, egy hordozható mobilalkalmazásra és az ezeket kiszolgáló központi API-ra.



1.1. ábra. A Synk szoftverarchitektúrája

1.2. Célközönség meghatározása

A szoftver célközönsége rétegzett, minden felhasználói típushoz dedikált felület tartozik:

- **Végfelhasználók (Vásárlók):** A Next.js felületen böngésznek, a React Native mobilalkalmazásban pedig tárolják és kezelik digitális jegyeiket.
- **Rendezvényszervezők:** A webfelületen / WPF kliensen keresztül végzik a komplex eseménymenedzsmentet és felhasználókezelést.
- **Helyszíni személyzet:** A mobilalkalmazás speciális funkcióival végzik a gyors, QR-kód alapú jegyérvényesítést.

Az automatikus tételbizonyítás egyik fontos megközelítése Herbrand nevéhez fűződik (1930). Herbrand algoritmust dolgozott ki olyan interpretáció előállítására, amely hamissá tehet egy formulát. Ugyanakkor, ha az illető formula logikailag igaz, tehát nincs olyan interpretáció, amelyben hamissá válna, ez az algoritmus véges sok lépés után megáll. [3] Ez a logikai szigor alapvető a Synk jegyfoglalási algoritmusánál is, ahol el kell kerülni az olyan állapotokat, amelyek ellentmondáshoz (pl. túlfoglaláshoz) vezetnének.

1.3. Alkalmazott technológiák és eszközök

Lásd: 7.1 melléklet ábrát.

- **Adatbázis:** PostgreSQL – A komplex relációs adatok és tranzakciók biztonságos kezeléséhez.
- **Backend:** RESTful API (ASP.NET Core) – A központi üzleti logika kiszolgálója.
- **Webes kliens:** Next.js – Szerveroldali rendereléssel támogatott React keretrendszer az optimális SEO és sebesség érdekében.
- **Mobil alkalmazás:** React Native (Expo) – Cross-platform megoldás, amely lehetővé teszi a natív élményt iOS és Android eszközökön a digitális jegyek kezeléséhez.
- **Asztali kliens:** WPF (C#) – Nagy teljesítményű adminisztrációs felület Windows környezetbe.

A rendszer logikai folyamatainak leírásához az alábbi szimbólumrendszert használjuk:

SZIMBÓLUM	**MAGYAR MEGNEVEZÉS**	**IDEGEN MEGNEVEZÉS**
$\wedge$	**ÉS**	**KONJUNKCIÓ**
$\vee$	**VAGY**	**DISZJUNKCIÓ**
$\Rightarrow$	**AKKOR..HA**	**IMPLIKÁCIÓ**
$\Leftrightarrow$	**AKKOR ÉS CSAK AKKOR**	**EKVIVALENCIA**

1.1. táblázat. Alkalmazott logikai műveletek

2. fejezet

Követelményanalízis

2.1. Funkcionális követelmények

A Synk rendszer funkcionális követelményei a tier-alapú jegyértékesítési modellre és a többrétegű kliensarchitektúrára fókuszálnak.

2.1.1. Webes felület (Next.js)

- **Események listázása:** A koncertek adatlapjának megjelenítése (leírás, dátum, fellépők).
- **Tier-alapú jegyválasztás:** A felhasználó választhat a különböző árkategóriájú és szolgáltatástartalmú jegytípusok (pl. Early Bird, Standard, VIP) közül a szabad készlet erejéig.
- **Jegylistázás:** Biztonságos jegykezelés, PDF formátumú jegyletöltés.

2.1.2. Mobil alkalmazás (React Native + Expo)

- **Jegyek kezelése:** A megvásárolt, különböző kategóriájú jegyek tárolása és vizuális megjelenítése.
- **QR-kódos validáció:** Egyedi, dinamikusan generált azonosítók megjelenítése a belépéshez.
- **Beléptető modul:** A személyzet számára elkülönített felület a jegyek szkennelésére és valós idejű érvénytelenítésére az API-n keresztül.

2.1.3. Adminisztrációs kliens (WPF)

A WPF alkalmazás kizárólag belső, adminisztratív célokat szolgál, közvetlen elérést biztosítva az üzleti adatokhoz:

– **Rendszerfelügyelet:** Felhasználó, és felhasználói jogosultságok kezelése és az adatbázis integritásának ellenőrzése.

2.2. Nem funkcionális követelmények

– **Tranzakcionális biztonság:** A PostgreSQL ACID tulajdonságait kihasználva garantálni kell, hogy a jegy-tierek darabszáma (stock) ne csökkenhessen negatív tartományba párhuzamos vásárlások esetén sem.

– **Keresőoptimalizálás (SEO):** A Next.js használatával biztosítani kell az eseményoldalak gyors indexelhetőségét.

– **Hordozhatóság:** A React Native kódalapnak minimális módosítással futnia kell mind iOS, mind Android eszközökön.

– **Adatvédelem:** A GDPR előírásoknak megfelelő titkosított adattárolás.

2.3. Használati esetek (Use Cases)

ID	Megnevezés	Aktor
UC-01	Jegyvásárlás adott kategóriában (Tier)	Vásárló
UC-02	Jegyérvényesítés mobil eszközzel	Személyzet
UC-03	Esemény és jegy-kvóták beállítása	Adminisztrátor
UC-04	Felhasználókezelés	Adminisztrátor

2.1. táblázat. A Synk platform használati esetei

2.3.1. Kiemelt Use Case: Jegyvásárlás (UC-01)

Leírás: A vásárló kiválasztja a kívánt jegy-tiert (pl. VIP), megadja a darabszámot, majd a rendszer ellenőrzi a szabad kontingenst az adatbázisban. **Logikai feltétel:** Let Q_{tier} be the current quota of a specific ticket category. `Requested_Amount <= Q_tier AND Payment_Successful => Success` Siker esetén a Q_{tier} értéke dekrementálódik a vásárolt darabszámmal.

3. fejezet

Rendszertervezés

3.1. Adatbázis terv

3.1.1. ER-diagram és kapcsolatok

Az ER-diagram a rendszer fő entitásait és azok kapcsolatát mutatja be. A tervezés során a normalizáció 3NF-ig történik, ahol indokolt (teljesítmény- vagy lekérdezési okokból) denormalizáció alkalmazható cache-n vagy read-model rétegben.

Fő entitások és kapcsolatok (röviden):

- **User** — felhasználói fiókok: 1-N kapcsolat az **Order**-rel és 1-N az **Ticket**-kel (ha a jegyek fel vannak használóhoz rendelve). A User tartalmaz jogosultsági információkat (role/permission), amelyek külön táblában normalizálhatók (**Role**, **UserRole**).
- **Event** — esemény: 1-N kapcsolat az **TicketType**-pal és az **EventArtist** / **EventVenue** kapcsolótáblákkal.
- **TicketType** — egy eseményhez tartozó jegytípusok (árkategória, kvóta, státusz). 1-N kapcsolat a **Ticket**-kel; tartalmaz mezőket: price, quota, reserved, visibility.
- **Order** — vásárlási rendelés: 1-N kapcsolat a **Ticket**-kel (minden rendelés több jegyet tartalmazhat). Order kapcsolódik **Payment**-hez (1:1 vagy 1:N a részletfizetések miatt).
- **Ticket** — egyedi jegy entitás: tartalmaz QR-kód token-t, státuszt (issued, used, refunded), kapcsolódik Order-hoz és (opcionálisan) User-hez.
- **Payment** — tranzakciós információk (providerId, status, amount, currency, timestamp).

– **Artist**, **Venue** — statikus referenciák; esemény összekapcsolása köztes táblákkal.

Kapcsolati jellemzők:

– Kulcsok: minden táblának UUID alapú primer kulcs (Postgres: UUID) — uniformitás és globális ütközésmentesség.

– Idegen kulcsokra ON DELETE RESTRICT/NO ACTION alapértelmezés, ahol az integritás fontos; soft-delete megoldás (isDeleted + deletedAt) az üzleti adatok megőrzésére.

– Indexelés: gyakori lekérdezések alapján (Event.id, TicketType.eventId, Ticket.qrToken, Order.userId, Payment.providerId), partiális indexek például csak aktív jegyekre.

3.1.2. Adatszótár és táblaleírások

A fontosabb táblák rövid leírása — oszlopnév (típus) — megjegyzések.

users

– id (UUID) — primer kulcs

– email (VARCHAR, unique) — bejelentkezéshez

– passwordHash (VARCHAR) — Argon2 tárolása

– displayName (VARCHAR)

– roleId (UUID) — idegen kulcs roles táblára (nem kötelező - Default: Customer)

– createdAt (TIMESTAMP WITH TIME ZONE)

– updatedAt (TIMESTAMP WITH TIME ZONE)

– deletedAt (TIMESTAMP WITH TIME ZONE) — soft delete

events

– id (UUID)

– title (VARCHAR)

– description (TEXT)

– startTime (TIMESTAMP WITH TIME ZONE)

– endTime (TIMESTAMP WITH TIME ZONE)

– venueId (UUID)

– createdAt, updatedAt

ticket$_{t}$*ypes*

id (UUID)

eventId (UUID) — FK -> events.id

name (VARCHAR)

price (NUMERIC(10,2))

quota (INT) — teljes kvóta

reserved (INT) — aktuálisan lefoglalt

remaining (INT) — redundáns mező, csak read-optimalizálásra, triggerrel/frissítéssel karbantartva

salesStart, salesEnd (TIMESTAMP)

createdAt, updatedAt

orders

– id (UUID)

– userId (UUID)

– totalAmount (NUMERIC(10,2))

– currency (VARCHAR)

– status (VARCHAR) — pending, paid, cancelled, refunded

– createdAt, updatedAt

tickets

– id (UUID)

– ticketTypeId (UUID)

– orderId (UUID)

– qrToken (VARCHAR, unique)

– status (VARCHAR) — issued, used, revoked

– usedAt (TIMESTAMP) — ebből következik validáció

– createdAt, updatedAt

3.2. Rendszerarchitektúra és komponensek

3.2.1. Általános architektúra

A Synk többrétegű, mikroserviz-szerű logikát követ, de a vizsgaprojekt érdekében monolitikus, modulárisan felépített backend (ASP.NET Core) került tervezésre, amelyet a következő logikai rétegekre bontunk:

- **API réteg** — REST API, kontroller réteg, DTO-kon keresztül kommunikál.
- **Service/Business réteg** — üzleti szabályokat, tranzakciókezelést, validációt tartalmazza.
- **Repository/Domain réteg** — adatbázis műveletek, Unit of Work, EF DbContext kezelése.
- **Client réteg** — Next.js webapp, React Native mobilapp, WPF admin kliens.
- **Perzisztencia** — PostgreSQL adatbázis, opcionálisan CQRS read-model cache (Redis) olvasási optimalizációhoz.
- **Aszinkron feldolgozás** — background jobok (pl. jegy-generálás, szinkronizáció külső szolgáltatóval), RabbitMQ vagy beépített háttérfeladatok.
- **FileStorage** — profilképek, jegy PDF-ek tárolása S3-kompatibilis tárolón (MinIO, AWS S3).
- **Monitoring/Logging** — centralizált loggyűjtés (Serilog -> Seq/ELK), metrika (Prometheus/Grafana).

3.2.2. Komponensek és felelősségek

- **API Gateway / Reverse Proxy** — kérelem irányítás, TLS termináció (Traefik).
- **Auth Service** — Session token alapú authentikáció, refresh token kezelés; jogosultságok RBAC modellben.
- **Mobil App** — alacsony késleltetésű jegyvalidáció; jegykezelés

Deployment szempontok: konténerizáció (Docker), környezetspecifikus konfiguráció (konfigurációk titkos kezelése: Vault/Environment variables), CI/CD pipeline (build, migrációk, smoke tesztek).

3.2.3. Skálázás és rendelkezésre állás

- Vertikális skálázás a kezdeti fázisban; horizontális skálázás stateless service-eknél (API), állapotot DB/Redis tárolja.
- Adatbázis: replikáció (read-replica) olvasási terhelés leválasztására; master node tranzakciókhoz.
- Failover tervek és backup stratégia: napi teljes mentés, bináris WAL archiválás a gyors helyreállításhoz.

3.3. ORM (Object-Relational Mapping)

3.3.1. Tervezési döntések

Az ORM használata (Entity Framework Core) gyors fejlesztést és típusbiztonságot nyújt. Az EF-et a következő szabályok mentén használjuk:

- **Explicit mapping** — minden entitást Fluent API-val konfigurálunk, elkerülve implicit konvenciókra való túlzott támaszkodást.
- **Azonnali vs késleltetett betöltés** — alapértelmezettként explicit Include használata nagy lekérdezéseknél; Lazy Loading csak jól ellenőrzött helyeken, hogy elkerüljük a N+1 problémákat.
- **Tranzakciókezelés** — minden kritikus művelet (pl. jegyvásárlás) explicit tranzakcióba kerül: DbContext.Database.BeginTransactionAsync() + commit/rollback.
- **Concurrency** — Optimistic Concurrency: minden fontos entitás (pl. TicketType) tartalmaz ‘row$_version$‘$(bytea/xmin/timestamp)mezőt, EFConcurrencyToken-$
$-kéntkezelve; konfliktuseseténvisszadobottDbUpdateConcurrencyExceptionkezeléseésretry$

3.3.2. Mapping és DTO stratégia

A belső domain entitások nem kerülnek közvetlenül a kliens felé. DTO-kat és automapper profilt használunk a lehetséges túlzott adatkinyerés elkerülésére, valamint a visszaadott JSON szerkezet stabilitásának biztosítására.

3.4. Entity Framework

3.4.1. DbContext és konfiguráció

A ‘ApplicationDbContext‘a per-request scope-ban kerül regisztrálásra (AddDbContext). A konfiguráció alapelvei:

- **Fluent API** minden táblához: kulcsok, indexek, check constraint-ek definiálása kód alapú migrációk során.
- **Migrations** — kódból létrehozott, verzionált migrációk használata (dotnet ef migrations add‘), migráció futtatása deployment pipeline részeként.
- **Query optimization** — AsNoTracking() read-only lekérdezésekhez, projection (Select új DTO-kra) a felesleges adatátvitel csökkentésére.
- **Raw SQL** — csak akkor alkalmazzuk, ha a LINQ nem tud hatékony, optimalizált SQL-t generálni (pl. komplex aggregációk), és az SQL-injection elleni védelem biztosított paraméterezéssel.

3.4.2. Teljesítmény- és karbantartási praktikák

- N+1 probléma elkerülése: Eager loading (Include) és batch lekérések, illetve explicit Select-projekciók.
- Indexek felülvizsgálata ‘EXPLAIN ANALYZE‘alapján, és migrációval indexek hozzáadása/eltávolítása.
- Long-running query-k monitorozása és timeoutok beállítása (CommandTimeout).

3.5. REST API végpontok tervezése

3.5.1. Erőforrás-orientált tervezés

Az API RESTful elveket követ: erőforrások URL-jei (pl. ‘/api/events/id‘, ‘/api/events/id/ticket-types‘) következetesek és intuitívak. Verziózás: Accept header-ben a jövőbeli visszafelé kompatibilitás miatt.

3.5.2. Példa végpontok és viselkedés

- GET /api/events‘— lista + pagination + filterek (dateRange, artistId) + sort.
- GET /api/events/id‘— részletes esemény adatok (ticketTypes nélkül vagy opcionálisan ?include=ticketTypes).

- ‘POST /api/orders/purchase‘— jegyvásárlás. Fontos: idempotencia kulcs támogatása (Idempotency-Key header) a duplikált fizetések elkerülésére. A végpont atomikus tranzakciót indít, ellenőrzi a kvótát és lefoglalja a jegyeket, majd kezdeményezi a fizetést.
- ‘PATCH /api/tickets/id/validate‘— jegy érvényesítés beléptető appból, audit log készítése minden érvényesítésről.

3.5.3. Hiba- és státuszkódok

Következetes HTTP státuszkódok és hibaválaszok:

- 200 OK — sikeres lekérés
- 201 Created — sikeres létrehozás
- 202 Accepted — aszinkron folyamat elfogadva (pl. külső fizetés feldolgozása)
- 400 Bad Request — validációs hibák; válasz részletes hibaobjektummal (‘field‘, ‘message‘)
- 401 Unauthorized, 403 Forbidden — auth/authz hibák
- 404 Not Found — nem létező erőforrás
- 409 Conflict — versenyhelyzet vagy idempotencia ütközés (pl. quota insuf.)
- 500+ — szerver oldali hibák (monitoring és alerting generálása)

3.5.4. Biztonság és adatvédelem

Minden érzékeny művelethez HTTPS kötelező. Session tokenek short-lived access token + long-lived refresh token mechanizmussal.

A jegyvásárlás kritikus pont; ajánlott megközelítések:

- **Pesszimista blokkolás**: SELECT ... FOR UPDATE a kvóta soron, ha a konkurencia nagyon magas és a tranzakciók rövidek.
- **Optimistic concurrency**: row$_{v}ersionellen\H{o}rzés; haversenyvan, atranzakcióvisszadobásrak$

- **Idempotencia**: belső vásárlás egységesen kezelhető idempotencia kulccsal, hogy duplikáció elkerülhető legyen.

3.6. UI/UX Tervek (Wireframes)

A wireframe-ek és UI specifikáció célja a felhasználói folyamatok minimalizálása és a hibalehetőségek csökkentése.

Fő elvek:

- **Egyszerűsített vásárlási folyamat** — 3 lépés: (1) esemény kiválasztás, (2) jegytípus darabszám, (3) fizetés és visszaigazolás/QR megjelenítés.
- **Responszív kialakítás** — Next.js + CSS-in-JS (pl. Tailwind/Emotion) a komponensek mobil/desktop változatára; WPF admin felület esetén rugalmas grid és skálázható vezérlők.
- **Átlátható hibakezelés** — valós idejű validáció (pl. darabszám > quota figyelmeztetés), inline hibauzenetek és globális toast értesítések.
- **Accessibility (a11y)** — releváns aria attribútumok, kontraszt, tab-navigáció, képernyőolvasó támogatás.
- **Beléptetés UX** — Mobil alkalmazás: minimalisztikus képernyő nagy, egyértelmű validáció gombokkal.

Wireframe forgatókönyvek:

1. **Vásárlói út**: keresés -> esemény lap -> jegyválasztó modal (valós időben elérhető mennyiség) -> fizetési űrlap -> visszaigazolás + jegy PDF/QR.
2. **Admin esemény létrehozás**: űrlap több lépcsőben (alapadatok, jegytípusok, kvóták, publikálás); előnézeti nézet a publikus oldallal.
3. **Beléptetési út**: kereső / QR scanner -> jegy megtalálása -> státusz ellenőrzés -> érvényesítés + audit log.

4. fejezet

Megvalósítás

4.1. Adatbázis réteg implementációja

4.2. REST Backend fejlesztése

4.2.1. Adatkezelés és Üzleti logika

4.2.2. Biztonság és Autentikáció (JWT)

4.3. React Frontend fejlesztése

4.3.1. Komponens-architektúra

4.3.2. Reszponzív megoldások és Mobil nézet

4.4. WPF Desktop alkalmazás

4.4.1. MVVM architektúra és Adatkötés

4.4.2. Törzsadat-karbantartó modulok

5. fejezet

Minőségbiztosítás és Tesztelés

5.1. Unit tesztek (Egységtesztelés)

5.2. API tesztelés (Postman/Swagger)

5.3. Felhasználói felület és Reszponzivitás tesztelése

5.4. Hibajegyzék és javítások

6. fejezet

Felhasználói útmutató

6.1. Rendszerkövetelmények és Telepítés

6.2. A szoftver használata (Web/WPF)

7. fejezet

Összegzés

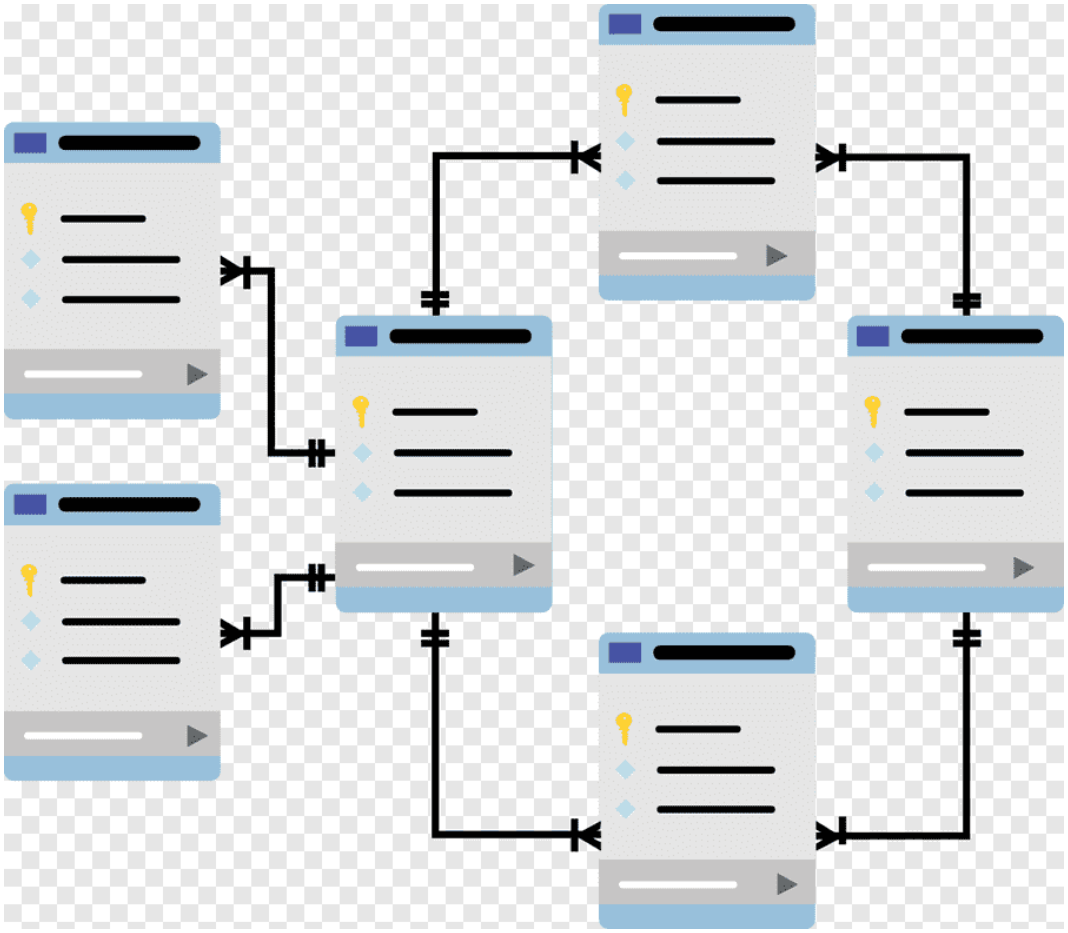
7.1. Projekt értékelése

7.2. Továbbfejlesztési lehetőségek

Köszönetnyilvánítás

Szeretnék köszönetet mondani mindazoknak, akik segítették a vizsgaremekem létrejöttét, külön megköszönve.....

Mellékletek



7.1. ábra. Adatbázis ábra

EZ EGY MELLÉKLET!!!

7.2. ábra. Szkennelt dokumentum

Irodalomjegyzék

[1] Fazekas István: *Valószínűségszámítás*, Debreceni Egyetem, Debrecen, 2004.

[2] Tómács Tibor: *A valószínűségszámítás alapjai*, Líceum Kiadó, Eger, 2005.

[3] Várterész Magdolna: *Az informatika logika alapjai előadások*, 2006/07-es tanév 1.félév, http://users.atw.hu/de-mi/index.php?r=file/download&id=219, Letöltve: 2019 március 12.-én